RUPEEK · ENGINEERING

# Maid Booking System

Machine Coding Round · Question B

Welcome, and thank you for taking the time. This is the first round of our SDE-3 process: a take-home, hands-on machine-coding exercise you will complete offline. The goal is not to finish every feature, but to show us how you structure a non-trivial system — clean modules, well-placed abstractions, and code another engineer could pick up and extend.

## 1 Round Overview

You will build a working backend service that models the domain described in Section 3. We are interested in the shape of your solution far more than its size — a small, clean, working slice is worth more than a large, tangled, half-working one.

**Format** Take-home exercise, completed offline in your own environment.
**Time window** Up to 2 days (48 hours) from when you receive this brief.
**Stack** Java 17+ with Spring Boot.
**Delivery** A runnable Spring Boot service plus a short README, shared as a repository or archive.
You do not need to use the full window — a focused, well-designed solution is worth more than a rushed, feature-complete one. Please plan your time so the core flows are clean and working, then use any remaining time for tests and polish.

## 2 What We Are Evaluating

We care much more about design and code quality than feature count. Prioritise a well-structured, working core over breadth. Concretely, we look for:

- **Working code —** it compiles, runs, and the core flows can be exercised through REST endpoints, tests, or a runnable entry point.
- **Modularity —** clear packages with single responsibilities; domain logic kept separate from framework and persistence concerns.
- **Abstractions & interfaces —** interfaces at the right seams; you program to contracts, not implementations.
- **Extensibility —** new types (property/maid kinds, payment methods, booking types) can be added with minimal, localised change — open for extension, closed for modification.
- **Domain modelling —** entities, value objects, and relationships that reflect the problem accurately, with well-defined state transitions.
- **Clean code —** readable naming, small cohesive methods, no duplication, consistent style.
- **Edge cases —** input validation, invalid state transitions, and concurrency on shared inventory.
- **Testing —** meaningful unit tests that pin down the core business rules.

## 3 Problem Statement

Design and implement the backend for a maid (home-services) booking platform. Customers discover maids and book them — instantly, on a schedule, or on a recurring basis; maids are onboarded onto the platform with their skills and availability. A booking is paid for and can later be cancelled.

Functional requirements:

### 3.1 Maid Discovery

- Search and browse maids by criteria such as locality, service type (for example cleaning, cooking, dishwashing), and required date or time.
- Support filters such as rating, price, skills, and gender preference — and design so that new filters can be added later without reworking the search.
- Return only maids who are actually available for the requested time.

### 3.2 Addition of Maid

- Onboard a new maid with profile details: name, location, skills or services offered, pricing, and availability windows.
- Model a maid's availability so the system can reason about free versus busy slots.

### 3.3 Booking a Maid

- Support three booking types behind a common abstraction — Instant (book a maid available right now), Scheduled (book for a specific future date and time), and Recurring (a repeating slot, for example every weekday at 9am or every Sunday).
- Check availability and prevent double-booking of the same maid for overlapping slots.
- Ensure adding a new booking type later requires minimal change.

### 3.4 Payment

- Take payment for a booking.
- Support multiple payment methods (for example card, UPI, wallet) behind a common abstraction, so new methods can be plugged in.
- For recurring bookings, consider how payment is handled per occurrence.

### 3.5 Cancellation

- Cancel an existing booking — for recurring bookings, support cancelling a single occurrence or the whole series.
- Apply a cancellation and refund policy — simple, but pluggable.
- Free up the maid's slot so it becomes discoverable and bookable again.

## 4 Technical Requirements

- Java 17+ and Spring Boot.
- Expose the core operations — discovery, add, book, pay, cancel. REST endpoints are preferred; a clean service layer exercised by tests is also acceptable.
- In-memory persistence (collections or H2) is fine — do not spend time on a production database — but keep it behind repository interfaces so it could be swapped later.
- Mock any third-party dependency (e.g. a payment gateway) behind your own abstraction.
- Include a README covering how to build and run, plus your key assumptions.

## 5 Deliverables

- A runnable Spring Boot project (source and build file).
- A short README: how to run, key design decisions, assumptions, and what you would do with more time.
- Unit tests for the core business logic.

## 6 Evaluation Rubric

The follow-up discussion will centre on your design choices and trade-offs. This is roughly how we weight the review:

| Area | What we look for | Weight |
|---|---|---|
| **Design & Abstractions** | Interfaces at the right seams, extensibility, SOLID, programming to interfaces | High |
| **Domain Modelling** | Accurate entities & value objects, correct relationships and state transitions | High |
| **Code Quality** | Readable naming, small cohesive methods, no duplication, consistent style | High |
| **Correctness** | Core flows work end-to-end; availability and double-booking handled | High |
| **Edge Cases** | Validation, invalid state transitions, concurrency on shared inventory | Medium |
| **Testing** | Meaningful unit tests around the core business logic | Medium |

## 7 Bonus — Only If Time Permits

- Concurrency handling for simultaneous bookings on the same inventory.
- A pluggable pricing / dynamic-pricing strategy.
- Idempotency for payment operations.
- Basic API documentation (OpenAPI / Swagger).

## 8 Out of Scope

Please do not spend time on these — depth of design beats breadth of features:

- Authentication / authorisation, and any UI or frontend.
- Production-grade persistence, migrations, or deployment.
- Exhaustive feature coverage.
- Real third-party gateway integration — mock it behind your abstraction.

## 9 Ground Rules & Submission

- You may use your IDE, language documentation, and standard libraries.
- Please write the code yourself. Be ready to walk through it, explain your trade-offs, and extend it live in the follow-up.

- When done, share the project (repo or archive) along with the README.

*Good luck — we are looking forward to seeing how you think.*